The S.A.F.E. Intent Framework

An RFC-Style Proposal for Validated Intent in AI-Assisted Administrative Automation

Rogel S.J. Corral
Independent Researcher

Draft v1.0
Status: Request for Comments

Date: 2026-02-18

Request for Comments

Feedback is requested on:

- Missing failure modes and edge cases
- Enforcement mechanisms that are practical in real operations workflows
- Thresholds for enumeration and confirmation in large target sets
- Integration profiles, especially identity and IaC workflows

Abstract

LLM-assisted administrative automation can generate scripts that are syntactically valid and operationally successful while being semantically incorrect. In privileged environments, this failure mode can create high blast radius changes with reduced friction and weaker review signals. The S.A.F.E. Intent Framework defines a minimum set of controls that must occur before privileged write actions execute, and the minimum evidence that must exist afterward. The goal is not to prove correctness of arbitrary code, but to reduce high impact intent mismatches through enforced enumeration, explicit confirmation, auditable execution output, and evidence grounded rollback.

1 Motivation

Administrative automation used to fail loudly. Tool friction, syntax mistakes, and environment quirks often prevented incorrect changes from executing at scale. LLM assistance changes the operating reality by making runnable artifacts cheap and fast to produce. That is useful, but it removes natural brakes. The core risk is not that the script fails. The core risk is that the script succeeds while doing the wrong thing.

S.A.F.E. restores deliberate verification steps before privileged writes. It is designed to be adoptable in real environments, including those that cannot afford heavy process overhead.

1.1 Design by Contract lineage

S.A.F.E. deliberately applies Design by Contract style discipline to privileged operational changes. In this framing, enumeration and snapshotting act as enforced preconditions, verification acts as postconditions, and the Minimal Evidence Record acts as an invariant that must hold for every

compliant run. This document acknowledges the strong influence of Bertrand Meyer's Design by Contract work and adapts its contract concepts from software component behavior to operational change execution [1, 2].

This is attribution. It does not imply endorsement by any cited author.

1.2 Real world anchors (optional, limit to two)

If you include real world incidents, include at most two and cite primary sources.

- Incident 1 (automation or privileged change blast radius): On 2017-01-31, GitLab.com suffered a major database incident leading to prolonged downtime and irreversible loss of several hours of production data, following a sequence of operational errors during recovery [3].
- Incident 2 (AI mediated automation risk): In July 2025, reports described a failure case in which an AI coding agent executed destructive actions including database deletion despite explicit constraints, highlighting the need for hard execution gates and rollback readiness [4].

2 Scope and non goals

2.1 In scope

S.A.F.E. applies when an LLM materially shapes an artifact that can change control plane state. This includes scripts, CLI sequences, IaC changes, and runbook steps that result in privileged writes.

2.2 Out of scope

- Proving functional correctness of arbitrary code
- Replacing existing change management
- Guaranteeing zero incidents
- Solving general hallucination outside action paths

S.A.F.E. is an execution gating and intent verification standard, not a knowledge correctness standard.

3 Terminology and normative keywords

The keywords MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY indicate requirement strength and are to be interpreted as described in RFC 2119 and RFC 8174.

3.1 Terms

- Write action: any create, update, delete, enable, disable, assign, revoke, or policy modification
- Target set: the objects affected by a write action
- Enumeration: a read-only step that produces the target set or a verifiable representation of it
- Echo output: a human-readable restatement of intended action, scope logic, and target count
- Snapshot: a before-state capture sufficient for verification and rollback
- Rollback plan: a recovery procedure referencing captured state, not generic suggestions
- MER: Minimal Evidence Record, the mandatory evidence bundle for a compliant run

4 Problem model

Privileged workflows fail catastrophically when intent is underspecified, scoping is vague, precedence rules are misunderstood, errors are suppressed, and rollback is assumed rather than proven. LLM assistance increases the probability that a runnable artifact exists, but does not guarantee that the artifact matches operator intent.

This RFC focuses on the gap between plausible output and verified intent.

5 Threat model

S.A.F.E. addresses two categories.

5.1 Unintentional failure modes

- Overbroad targeting due to vague scope
- Wrong predicate with valid syntax
- Inheritance and precedence mistakes
- Silent partial failures that produce false success cues
- Non idempotent effects that compound on rerun
- Secrets exposed via logs or artifacts
- Rollback fiction that fails under pressure

5.2 Adversarial misuse

- Social engineering that pushes an operator to execute LLM generated privileged actions
- Retrieval poisoning and indirect prompt injection that influences action plans [8]
- Evidence tampering, selective logging, or missing logs

6 The S.A.F.E. requirements

S.A.F.E. defines four mandatory control families. Each family has requirements and mandatory evidence artifacts.

6.1 S: Separation of context

Objective: prevent unsafe coupling between untrusted inputs and privileged execution.

- SAFE-REQ-S1: When feasible, privileged credentials and write-capable tools MUST be inaccessible during plan generation.
- SAFE-REQ-S2: Enumeration and snapshot collection MUST occur before any write action.
- SAFE-REQ-S3: If retrieval is used, retrieved content MUST be treated as untrusted evidence, not as executable instruction [8].

Mandatory evidence:

- Phase boundaries recorded, including toolchain identity per phase
- Any exception to SAFE-REQ-S1 documented in MER with justification

6.2 A: Ambiguity resolution

Objective: force explicit scope confirmation, stop on underspecified intent.

- SAFE-REQ-A1: Before a write action, the system MUST enumerate the target set or produce a verifiable representation.
- SAFE-REQ-A2: The resolved filter logic MUST be printed, and MUST be identical between enumeration and write phases.
- SAFE-REQ-A3: Operator confirmation MUST occur after enumeration and before write. Implementations MUST support confirmation thresholds for high-scale operations, including a second human approver when the target count or affected-fleet percentage exceeds configured limits.
- SAFE-REQ-A4: If ambiguity remains or conflicts are detected, execution MUST stop and request clarification or escalate.

Mandatory evidence:

- Target count
- Scope logic as executed
- Stable sample identifiers or stable deterministic ordering
- Operator confirmation record

6.3 F: Forensic idempotency

Objective: prevent silent drift, make actions observable and repeat-safe.

- SAFE-REQ-F1: Scripts MUST report structured per-target status and MUST NOT suppress errors without structured reporting.
- SAFE-REQ-F2: Scripts SHOULD implement state checks to avoid mutation when desired state is already present.
- SAFE-REQ-F3: Runs MUST produce after-state verification output, such as a diff or validation query.
- SAFE-REQ-F4: Evidence artifacts MUST NOT contain secrets. Secret scanning SHOULD be applied to artifacts and logs.

Mandatory evidence:

- Per-target success and failure summary
- Before and after deltas or verification query output
- Secret scan status and redaction status

6.4 E: Evidence based rollback

Objective: ensure rollback is grounded in captured state.

- SAFE-REQ-E1: Rollback plans MUST reference snapshot identifiers and captured state values.
- SAFE-REQ-E2: Rollback MUST be executable without LLM assistance. For write actions above configured risk thresholds, the rollback script or procedure MUST be generated, reviewed, and stored alongside the primary action artifact, not as a later best-effort plan.
- SAFE-REQ-E3: If rollback cannot be made reliable, execution MUST require sandboxing or escalation.

Mandatory evidence:

- Snapshot references
- Rollback steps tied to captured values
- Restore validation check

7 Compliance levels (normative definitions)

These are definitions of what an implementation enforces.

- SAFE-L0: Documentation only. No automated enforcement.
- SAFE-L1: CI gating. The pipeline MUST verify presence of required phases and MER artifacts.
- SAFE-L2: Runtime enforcement. A wrapper MUST enforce phase ordering, MUST block writes without SAFE-REQ-A3 confirmation, and MUST generate MER.
- SAFE-L3: High risk class enforcement. SAFE-L2 plus sandboxing when feasible, immutable evidence storage, and canary rollout requirements for defined high blast radius change classes.

Implementations MUST state the compliance level per environment and per change class.

8 Traceability Matrix (compact)

This matrix links failure modes to requirements and evidence. The expanded matrix appears in Appendix A.

ID	Failure mode	Control	Requirement refs	Evidence
FM-01	Overbroad targeting	A	A1, A2, A3	Enumeration output, scope logic, confirmation
FM-02	Wrong predicate, valid syntax	A, F	A2, F3	Predicate output, after verification
FM-03	Silent partial failure	F	F1	Per-target status, failure counts
FM-04	Non idempotent rerun drift	F	F2, F3	State checks, after diff
FM-05	Secret exposure	S, F	S1, F4	Phase boundary, secret scan
FM-06	Rollback fiction	E	E1, E2	Snapshot refs, restore check
FM-07	Indirect prompt injection	S, A	S3, A4	Untrusted retrieval handling, stop record
FM-08	Citation laundering	A, F	A3, F3	Confirmation, actual diff evidence

9 Minimal Evidence Record

A compliant run MUST generate an MER containing:

- Intent statement and timestamp
- Operator identity or approval mechanism
- Compliance level, environment, change class
- Toolchain versions and wrapper version
- Enumeration artifact and target count
- Snapshot references

- Execution summary, success and failure counts
- After-state verification output
- Rollback plan reference
- Secret scan status and redaction status
- Any exceptions, such as inability to sandbox

A suggested MER schema appears in Appendix B.

10 Related work (short)

10.1 Design by Contract

Design by Contract formalizes reliability through explicit preconditions, postconditions, and invariants treated as enforceable obligations between caller and supplier [1, 2]. S.A.F.E. adapts this contract framing from code-level behavior to operational behavior for privileged changes, adding mandatory evidence capture and rollback grounding to support audit, recovery, and controlled execution when LLM generated artifacts are involved.

10.2 AI risk management and prompt injection

S.A.F.E. is compatible with risk framing such as the NIST AI Risk Management Framework [5]. When retrieval is present, prompt injection and indirect injection risks motivate treating retrieved content as untrusted evidence [8].

10.3 Prior work by the author

S.A.F.E. consolidates and standardizes mitigations motivated by the author's prior analyses of LLM-generated privileged scripts and retrieval-augmented systems [9, 10].

11 Security considerations

Evidence artifacts should be tamper-resistant, secrets should be prevented from entering logs and should be scanned and redacted, retrieval corpora should be treated as untrusted input boundaries, and privileged credentials should be isolated from generation contexts when feasible.

12 Limitations

S.A.F.E. does not prove correctness of arbitrary code. It reduces high impact intent mismatch probability by enforcing enumeration, confirmation, evidence capture, and rollback reliability. Human judgement remains part of the safety story, especially for complex change classes.

A Expanded Traceability Matrix

ID	Failure mode	Example scenario	Impact	Control	Required evidence
FM-01	Overbroad targeting	Wildcard matches all users instead of one department	High	A	Target count, executed filter, confirmation

FM-02	Under-scoped targeting	Pagination or API defaults omit targets	Medium	A, F	Enumeration with pagination proof, after verification
FM-03	Wrong predicate, valid syntax	Disable MFA on wrong group attribute	High	A, F	Echoed predicate, after validation
FM-04	Hidden precedence and inheritance	Nested groups re-grant privileges	High	F, E	Before/after effective permissions snapshots
FM-05	Silent partial failure	Errors suppressed, success reported	High	F	Per-target status, failure counts
FM-06	Non idempotent rerun drift	Duplicate rules appended on rerun	Medium	F	State checks, post-run diff
FM-07	Unsafe defaults	Provider defaults widen effect	High	A	Resolved parameters, stop record
FM-08	Environment drift	Targets change after enumeration	Medium	S, A	Enumeration timestamp, TTL, re-enum on mismatch
FM-09	Concurrent changes	Two operators collide	Medium	S, F	Snapshot ids, after verification
FM-10	Unreliable rollback	Undo assumes state never captured	High	E	Snapshot refs, restore check
FM-11	Rollback requires LLM	Recovery invented on the fly	High	E	Human runnable rollback steps, escalation record
FM-12	Secrets in output	Token printed to logs	High	S, F	Secret scan, redaction proof
FM-13	Sensitive data in evidence	MER captures secrets or PHI	High	F	Data class tags, redaction status
FM-14	Vague intent	"Clean up old accounts" triggers deletes	High	A	Stop on ambiguity, clarification request
FM-15	Citation laundering	Script cites policy but violates it	Medium	A, F	Actual diff evidence, not citations
FM-16	Conflicting retrieval sources	Two runbooks disagree	Medium	A	Conflict detection, stop event
FM-17	Indirect prompt injection	Retrieved doc injects malicious instructions	High	S, A	Untrusted retrieval handling, stop record
FM-18	Retrieval poisoning	KB modified to shape actions	High	S	Corpus provenance tag, change history refs
FM-19	Hallucinated names	Model invents group or parameter	Medium	A	Not-found mismatch record, stop event
FM-20	Count mismatch	Enumerate count differs from write count	Medium	A	Count comparison, re-confirmation record
FM-21	Permission mismatch	Script runs with broader rights than needed	High	S	Role used, credential scope evidence
FM-22	Unbounded retries	Retries cause repeated writes	Medium	F	Retry policy, mutation counts
FM-23	No canary rollout	Full rollout without containment	High	L3	Canary plan evidence, staged output
FM-24	Operator fatigue	Confirmation becomes click-through	Medium	A	Confirmation summary, risk tier
FM-25	Missing post verification	Change executed but not validated	High	F	Verification query output
FM-26	Evidence tampering	Logs edited after incident	High	L3	Immutable evidence store ref
FM-27	Selective logging	Only successes recorded	Medium	F	Both success and failure counts
FM-28	Unsafe exception path	Sandbox bypass not recorded	Medium	S, E	Exception record, compensating controls
FM-29	Unclear ownership	No accountable operator	Medium	MER	Operator identity, approval mechanism

FM-30	Unclear intent statement	Intent missing or fluff	Medium	MER, A	Intent captured, ambiguity stop

B MER schema (suggested)

This appendix provides a suggested Minimal Evidence Record schema. Implementations MAY use an equivalent structured format.

```
{
  "mer_version": "0.4",
  "safe_compliance_level": "L2",
  "run_id": "uuid-or-hash",
  "timestamp_utc": "2026-02-18T00:00:00Z",
  "environment": {
    "name": "prod",
    "change_class": "iam|tenant_policy|endpoint_mass|network_access|other",
    "risk_tier": "low|medium|high"
  },
  "intent": {
    "operator_intent_text": "Human written intent statement",
    "requested_action": "create|update|delete|assign|revoke|enable|disable|other",
    "justification": "Why this change is needed",
    "approval_ticket_ref": "optional change ticket reference"
  },
  "operator": {
    "actor_id": "human or service identity",
    "approval_mechanism": "interactive_confirm|two_person_rule|ticket_approval|other",
    "approver_ids": ["optional"]
  },
  "toolchain": {
    "wrapper_name": "safe-wrapper",
    "wrapper_version": "x.y.z",
    "llm_provider": "optional",
    "llm_model": "optional",
    "execution_tools": ["powershell", "bash", "terraform", "az", "aws", "gcloud", "other"]
  },
  "phases": {
    "generation": {
      "write_credentials_present": false,
      "retrieval_used": false,
      "retrieval_provenance": "optional corpus identifiers"
    },
    "enumeration": {
      "enumeration_time_utc": "2026-02-18T00:00:00Z",
      "scope_logic_executed": "Exact filter or query used",
      "target_count": 0,
      "target_representation": {
```

```
      "method": "full_list|hashed_list|stable_sample",
      "stable_sample_ids": ["optional"],
      "hashed_list_digest": "optional"
    }
  },
  "snapshot": {
    "snapshot_time_utc": "2026-02-18T00:00:00Z",
    "snapshot_ids": ["id1", "id2"],
    "snapshot_definition": "What was captured and why it is sufficient"
  },
  "confirmation": {
    "confirmed": true,
    "confirmation_time_utc": "2026-02-18T00:00:00Z",
    "confirmation_summary_shown": "Echo output shown to operator",
    "threshold_flags": ["optional warnings triggered"]
  },
  "execution": {
    "execution_time_utc": "2026-02-18T00:00:00Z",
    "write_action_performed": true,
    "success_count": 0,
    "failure_count": 0,
    "per_target_status_ref": "pointer to stored artifact",
    "exit_policy": "fail_on_any_failure|allow_partial|other",
    "retries_policy": "bounded|none|other"
  },
  "verification": {
    "verification_time_utc": "2026-02-18T00:00:00Z",
    "verification_query": "Query or method used",
    "verification_output_ref": "pointer to stored artifact",
    "diff_summary": "Optional summary of before after deltas"
  },
  "rollback": {
    "rollback_plan_ref": "pointer to rollback plan artifact",
    "rollback_without_llm": true,
    "restore_validation_query": "How rollback success is validated"
  }
},
"security": {
  "secret_scan_performed": true,
  "secret_scan_tool": "gitleaks|trufflehog|other",
  "secret_scan_result": "pass|fail",
  "redaction_performed": true,
  "redaction_notes": "optional"
},
"evidence_storage": {
  "storage_type": "filesystem|object_store|immutable_log|ticket_system",
  "immutability": "none|append_only|immutable",
  "evidence_bundle_ref": "pointer to stored evidence bundle"
```

```
  },
  "exceptions": [
    {
      "exception_code": "NO_SANDBOX_AVAILABLE|OTHER",
      "justification": "Why this exception was necessary",
      "compensating_controls": ["canary", "two_person_rule", "short_ttl", "other"]
    }
  ]
}
```

C Integration profiles (draft)

This appendix will define practical profiles for Shell and PowerShell, Cloud CLIs, IAM platforms, IaC plan/apply workflows, and CI/CD pipelines, including two worked examples (one PowerShell, one IaC) that produce a sample MER.

D IANA considerations

This document has no IANA actions.

References

[1] B. Meyer, Applying "Design by Contract," *Computer*, 25(10):40–51, 1992. doi:10.1109/2.161279.

[2] B. Meyer, *Object-Oriented Software Construction*, 2nd edition, Prentice Hall, 1997.

[3] GitLab, Postmortem of database outage of January 31, 2017. https://about.gitlab.com/blog/postmortem-of-database-outage-of-january-31/.

[4] The Register, Report on Replit incident involving AI agent and database deletion, July 2025. https://www.theregister.com/2025/07/21/replit_saastr_vibe_coding_incident/.

[5] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, 2023. doi:10.6028/NIST.AI.100-1.

[6] S. Bradner, "Key words for use in RFCs to Indicate Requirement Levels," RFC 2119, 1997. Available: https://www.rfc-editor.org/rfc/rfc2119.

[7] B. Leiba, "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words," RFC 8174, 2017. Available: https://www.rfc-editor.org/rfc/rfc8174.

[8] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and Benchmarking Prompt Injection Attacks and Defenses," arXiv:2310.12815, 2023. doi:10.48550/arXiv.2310.12815. Available: https://arxiv.org/abs/2310.12815.

[9] R. S. J. Corral, "The Risk of LLM-Generated Administrative Scripts in Privileged Environments," Zenodo, Feb. 19, 2026. doi:10.5281/zenodo.18718481.

[10] R. S. J. Corral, "RAG Is Not a Safety System: Why Retrieval Does Not Solve AI Reliability," Zenodo, Feb. 22, 2026. doi:10.5281/zenodo.18728300.